LAB 7: FUZZ TESTING FOR INPUT VALIDATION BUGS

Software Engineering Lab Report — Automated Property-Based Testing with Hypothesis

Student Name: Ibrahim Khaleel	**Course:** Software Engineering Lab (SE)
SRN: PES2UG25CS809	**Topic:** Property-Based Fuzz Testing (Hypothesis)
Section / Sem: Section D, 5th Sem	**Module Assigned:** L6_L7_58 (Restaurant Bill Parser)

1. Objective & Learning Outcomes

Lab Objective: To explore fuzz testing as an automated security and quality assurance technique by discovering, analyzing, and resolving input validation vulnerabilities in a Python application module using the Hypothesis property-based testing framework.

- **Understand Fuzzing Methodology:** Comprehend how feeding pseudo-random, malformed, and non-canonical data uncovers unforeseen edge cases and crash conditions.
- **Property-Based Testing with Hypothesis:** Formulate behavioral invariants and domain constraints using @given decorators and strategy generators (st.text(), st.integers(), st.one_of()).
- **Test-Case Shrinking Analysis:** Leverage Hypothesis's automated shrinking engine to isolate minimal reproducible counterexamples for each failure.
- **Defensive Programming:** Implement defensive guards (type validation, boundaries, sanitization, zero-division guards) ensuring the module never experiences unhandled exceptions.

2. Conceptual Overview: Traditional Unit Testing vs. Property Fuzzing

Traditional unit tests test *known knowns* — scenarios developer intuition anticipates. However, software in production encounters unexpected character sets, boundary values, whitespace anomalies, and data-type mismatches. Hypothesis tests *properties* (invariants that must hold across thousands of randomly generated inputs). When a property fails, Hypothesis systematically simplifies the input to the simplest failing case.

Aspect	Traditional Unit Testing	Hypothesis Property-Based Fuzzing
Input Selection	Hardcoded exemplary inputs (e.g., '100,200', '1200/4')	Automated generation of thousands of randomized permutations
Coverage Scope	Covers happy paths and anticipated boundary values	Surfaces bizarre edge cases (Unicode spaces, empty delimiters, negative numbers, extreme integers)
Debugging Aid	Reports assertion failure on arbitrary hardcoded sample	Automated shrinking algorithm finds smallest reproducible trigger

3. Step-by-Step Lab Execution & Screenshots

Step 2: Initial Run of Buggy Module (Screenshot SS1)

The baseline application restaurant_bill_parser.py was executed using standard well-formed inputs. Under ideal inputs (paneer tikka, 250,100,240, and 1200/4), the program functions correctly because it assumes inputs are always clean and valid. However, it lacks defensive checks for types, empty fields, or edge cases.

```
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): paneer tikka
Dish name: Paneer Tikka

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,100,240
Item prices: [250, 100, 240]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/4
Each person pays: 300
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7>
```

Figure 1 (SS1): Baseline execution of buggy restaurant_bill_parser.py with standard inputs.

Step 3 & 4: Hypothesis Fuzz Testing Exposing Application Bugs (Screenshot SS2)

A comprehensive property-based fuzz test suite was created in test_restaurant_bill_parser.py using Hypothesis strategies (st.text(), st.integers(), st.one_of(), st.sampled_from()). When executed via python -m pytest test_restaurant_bill_parser.py -v on the unhardened module, Hypothesis rapidly identified **7 distinct critical failures** across all three functions.

```
test_restaurant_bill_parser.py::test_clean_dish_name_removes_special_characters PASSED [  6%]
test_restaurant_bill_parser.py::test_clean_dish_name_rejects_invalid_types FAILED [ 13%]
test_restaurant_bill_parser.py::test_clean_dish_name_rejects_empty_names FAILED [ 20%]
test_restaurant_bill_parser.py::test_parse_item_prices_valid_inputs PASSED [ 26%]
test_restaurant_bill_parser.py::test_parse_item_prices_handles_whitespace PASSED [ 33%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_invalid_values PASSED [ 40%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_negative_prices FAILED [ 46%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_empty_input PASSED [ 53%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_empty_elements PASSED [ 60%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_invalid_types FAILED [ 66%]
test_restaurant_bill_parser.py::test_split_amount_valid_inputs PASSED [ 73%]
test_restaurant_bill_parser.py::test_split_amount_rejects_negative_total FAILED [ 80%]
test_restaurant_bill_parser.py::test_split_amount_rejects_zero_or_negative_people FAILED [ 86%]
test_restaurant_bill_parser.py::test_split_amount_rejects_invalid_formats PASSED [ 93%]
test_restaurant_bill_parser.py::test_split_amount_rejects_invalid_types FAILED [100%]
```

Figure 2 (SS2 - Part A): Verbose pytest output showing 7 failing tests on the unhardened module.

```
restaurant_bill_parser.py:23: AttributeError
=========================== short test summary info ============================
FAILED test_restaurant_bill_parser.py::test_clean_dish_name_rejects_invalid_types - AttributeError: 'NoneType' object has no attribute 'strip'
FAILED test_restaurant_bill_parser.py::test_clean_dish_name_rejects_empty_names - Failed: DID NOT RAISE ValueError
FAILED test_restaurant_bill_parser.py::test_parse_item_prices_rejects_negative_prices - Failed: DID NOT RAISE ValueError
FAILED test_restaurant_bill_parser.py::test_parse_item_prices_rejects_invalid_types - AttributeError: 'NoneType' object has no attribute 'split'
FAILED test_restaurant_bill_parser.py::test_split_amount_rejects_negative_total - Failed: DID NOT RAISE ValueError
FAILED test_restaurant_bill_parser.py::test_split_amount_rejects_zero_or_negative_people - BaseExceptionGroup: Hypothesis found 2 distinct failures. (2 sub-exc
eptions)
FAILED test_restaurant_bill_parser.py::test_split_amount_rejects_invalid_types - AttributeError: 'NoneType' object has no attribute 'split'
========================= 7 failed, 8 passed in 6.89s ==========================
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7> python -m pytest test_restaurant_bill_parser.py -v
```

Figure 3 (SS2 - Part B): Short test summary confirming unhandled AttributeError, unhandled negative cases, and missing exceptions (7 failed, 8 passed in 6.89s).

Step 5: Execution of Fixed & Hardened Module (Screenshot SS3)

The application script was refactored with comprehensive defensive validation: type enforcement via isinstance(..., str), whitespace/marker stripping checks, empty string rejections, non-negative integer assertions, and split token format verification. Running python restaurant_bill_parser.py confirmed that the application preserves full functionality on valid user inputs while gaining resilience against corrupt inputs.

```
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7> python restaurant_bill_parser.py
==== Running clean_dish_name ====
Enter a dish name from the menu (e.g. *paneer tikka*): paneer
Dish name: Paneer

==== Running parse_item_prices ====
Enter item prices separated by commas (e.g. 250,180,320): 250,180,320
Item prices: [250, 180, 320]

==== Running split_amount ====
Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): 1200/4
Each person pays: 300
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7>
```

Figure 4 (SS3): Interactive run of the hardened restaurant_bill_parser.py demonstrating normal operation.

Step 6: Re-Running Hypothesis Fuzz Suite on Fixed Module (Screenshot SS4)

The Hypothesis test suite was re-run against the hardened module using python -m pytest test_restaurant_bill_parser.py -v. All **15 property-based tests passed completely** (100% pass rate in 1.70 seconds), confirming that the module now handles all arbitrary, extreme, and malformed inputs deterministically without crashing.

```
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7> python -m pytest test_restaurant_bill_
parser.py -v

test_restaurant_bill_parser.py::test_clean_dish_name_removes_special_characters PASSED          [  6%]
test_restaurant_bill_parser.py::test_clean_dish_name_rejects_invalid_types PASSED              [ 13%]
test_restaurant_bill_parser.py::test_clean_dish_name_rejects_empty_names PASSED                [ 20%]
test_restaurant_bill_parser.py::test_parse_item_prices_valid_inputs PASSED                     [ 26%]
test_restaurant_bill_parser.py::test_parse_item_prices_handles_whitespace PASSED               [ 33%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_invalid_values PASSED           [ 40%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_negative_prices PASSED          [ 46%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_empty_input PASSED              [ 53%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_empty_elements PASSED           [ 60%]
test_restaurant_bill_parser.py::test_parse_item_prices_rejects_invalid_types PASSED            [ 66%]
test_restaurant_bill_parser.py::test_split_amount_valid_inputs PASSED                          [ 73%]
test_restaurant_bill_parser.py::test_split_amount_rejects_negative_total PASSED                [ 80%]
test_restaurant_bill_parser.py::test_split_amount_rejects_zero_or_negative_people PASSED       [ 86%]
test_restaurant_bill_parser.py::test_split_amount_rejects_invalid_formats PASSED               [ 93%]
test_restaurant_bill_parser.py::test_split_amount_rejects_invalid_types PASSED                 [100%]

============================================ 15 passed in 1.70s ============================================
(venv) PS D:\CLG\SE\SE-LABS-PES2UG25CS809\lab7\L6_L7_58_restaurant_bill\LAB7>
```

Figure 5 (SS4): Pytest execution report confirming all 15 property-based fuzz tests PASSED on the fixed module.

4. Vulnerability & Edge Case Analysis

Fuzzing uncovered 6 fundamental vulnerabilities in the unhardened codebase. Below is a structured analysis of each bug, its root cause, the minimal failing counterexample found by Hypothesis shrinking, and the defensive fix applied:

Target Function	Vulnerability Uncovered	Shrunk Counterexample	Defensive Fix Implemented
clean_dish_name	Unhandled NoneType or non-string object caused AttributeError: 'NoneType' object has no attribute 'strip'.	name=None	Added if not isinstance(name, str): raise TypeError(...)
clean_dish_name	Empty string or strings containing only marker characters (*, #, ~) returned empty string instead of rejecting.	name='' name='*#~'	Checked if not name: raise ValueError('Dish name cannot be empty') after sanitization.
parse_item_prices	Missing type check crashed on non-string input. Negative numbers were parsed without raising errors.	text=None text='-1'	Enforced string type check; verified if value < 0: raise ValueError(...).
parse_item_prices	Malformed lists with adjacent commas or trailing commas caused ValueError: invalid literal for int() with base 10: ''.	text='100,,200'	Trimmed whitespace and explicitly checked if not price: raise ValueError(...).
split_amount	Zero diners caused unhandled ZeroDivisionError; negative diners or bill totals resulted in mathematically nonsensical splits.	text='1200/0' text='-500/2'	Validated total >= 0 and strictly required people > 0; raised ValueError.
split_amount	Missing or multiple slash delimiters caused ValueError: not enough values to unpack.	text='1200/4/2' text='1200'	Verified len(parts) == 2 before attempting integer conversion.

5. Answers to Lab Reflection Questions

Q1: What are the primary differences between traditional example-based unit testing and property-based fuzz testing?

Answer: Traditional unit testing relies on specific hardcoded input/output pairs designed according to the engineer's assumptions. It verifies that known cases work as expected. In contrast, property-based fuzz testing specifies general invariants (e.g., 'the function must never crash regardless of input', 'the output list must have non-negative integers') and generates hundreds of pseudo-random, edge-case, and boundary inputs. It excels at discovering 'unknown unknowns' that human test authors rarely consider, such as empty delimiters, extreme numeric boundaries, and non-ASCII character inputs.

Q2: How does Hypothesis test-case minimization (shrinking) assist developers in debugging?

Answer: When a randomized test fails on an unwieldy input (e.g., an 80-character string with complex Unicode characters), finding the root cause manually is difficult. Hypothesis automatically executes a shrinking algorithm, systematically reducing the input complexity (removing characters, zeroing numbers) while re-evaluating the test until it identifies the minimal reproducible counterexample (e.g., shrinking to None, '', or 0). This directly isolates the exact boundary condition causing the crash.

Q3: Why is defensive programming essential for software security and system reliability?

Answer: In production systems, unexpected or malformed user inputs can lead to uncaught exceptions, Denial of Service (DoS) crashes, or security exploits (such as buffer corruption or injection attacks). By adopting defensive programming—explicitly checking input data types, stripping and validating strings, bounding numeric ranges, and handling exceptional cases gracefully—the application establishes robust failure boundaries and prevents application downtime.

6. Source Code Appendices

Appendix A: Fixed Application Script (restaurant_bill_parser.py / processor.py)

The complete hardened application module code is divided into functional components below:

A.1 Dish Name Sanitizer Function:

```
def clean_dish_name(name):
    """Removes chef's-special markers and title-cases the dish name. Rejects invalid types and empty dish
names."""
    if not isinstance(name, str):
        raise TypeError("Dish name must be a string")
    name = name.strip()
    for ch in ['*', '#', '~']:
        name = name.replace(ch, '')
    name = name.strip()
    if not name:
        raise ValueError("Dish name cannot be empty")
    return name.title()
```

A.2 Price List Parser Function:

```
def parse_item_prices(text):
    """Parses comma-separated prices into a list of non-negative integers."""
    if not isinstance(text, str):
        raise TypeError("Prices must be provided as a string")
    if not text.strip():
        raise ValueError("Price list cannot be empty")
    prices = []
    for price in text.split(','):
        price = price.strip()
        if not price:
            raise ValueError("Price cannot be empty")
        try:
            value = int(price)
        except ValueError:
            raise ValueError("Prices must be valid whole numbers")
        if value < 0:
            raise ValueError("Prices cannot be negative")
        prices.append(value)
    return prices
```

A.3 Bill Split Calculation Function:

```
def split_amount(text):
    """Splits TOTAL/PEOPLE and calculates each person's share."""
    if not isinstance(text, str):
        raise TypeError("Bill details must be a string")
    parts = text.split('/')
    if len(parts) != 2:
        raise ValueError("Format must be TOTAL/PEOPLE")
    try:
        total = int(parts[0].strip())
        people = int(parts[1].strip())
    except ValueError:
        raise ValueError("Total and people must be whole numbers")
    if total < 0:
        raise ValueError("Total cannot be negative")
    if people <= 0:
        raise ValueError("Number of people must be positive")
    return total // people
```

A.4 Main Execution Routine:

```
def main():
    print("==== Running clean_dish_name ====")
    value = input("Enter a dish name from the menu (e.g. *paneer tikka*): ")
    print("Dish name:", clean_dish_name(value))

    print("\n==== Running parse_item_prices ====")
    value = input("Enter item prices separated by commas (e.g. 250,180,320): ")
    print("Item prices:", parse_item_prices(value))

    print("\n==== Running split_amount ====")
    value = input("Enter the bill and number of people as TOTAL/PEOPLE (e.g. 1200/4): ")
    print("Each person pays:", split_amount(value))

if __name__ == "__main__":
    main()
```

Appendix B: Updated Hypothesis Test Suite (test_restaurant_bill_parser.py / test_processor.py)

The complete 15 property-based fuzz tests developed using Hypothesis are partitioned by test target below:

B.1 Test Suite Setup & Module Imports:

```
import pytest
from hypothesis import given, strategies as st
from restaurant_bill_parser import clean_dish_name, parse_item_prices, split_amount
```

B.2 Property Tests for clean_dish_name:

```
# TEST 1: clean_dish_name()
@given(st.text(min_size=1).filter(lambda s: bool(s.strip().replace('*', '').replace('#', '').replace('~',
'').strip())))
def test_clean_dish_name_removes_special_characters(name):
    result = clean_dish_name(name)
    expected = name.strip()
    for ch in ['*', '#', '~']:
        expected = expected.replace(ch, '')
    assert result == expected.title()

@given(st.one_of(st.none(), st.integers(), st.floats(), st.lists(st.integers())))
def test_clean_dish_name_rejects_invalid_types(name):
    with pytest.raises(TypeError):
        clean_dish_name(name)

@given(st.sampled_from(["", " ", "*", "#", "~", "*#~"]))
def test_clean_dish_name_rejects_empty_names(name):
    with pytest.raises(ValueError):
        clean_dish_name(name)
```

B.3 Property Tests for parse_item_prices:

```
# TEST 2: parse_item_prices()
@given(st.lists(st.integers(min_value=0, max_value=100000), min_size=1, max_size=20))
def test_parse_item_prices_valid_inputs(prices):
    text = ",".join(map(str, prices))
    assert parse_item_prices(text) == prices

@given(st.lists(st.integers(min_value=0, max_value=100000), min_size=1, max_size=10))
def test_parse_item_prices_handles_whitespace(prices):
    text = ", ".join(map(str, prices))
    assert parse_item_prices(text) == prices

@given(st.text().filter(lambda s: not s.strip().isdigit()))
def test_parse_item_prices_rejects_invalid_values(value):
    with pytest.raises(ValueError):
        parse_item_prices("100," + value)

@given(st.integers(max_value=-1))
def test_parse_item_prices_rejects_negative_prices(price):
    with pytest.raises(ValueError):
        parse_item_prices(str(price))

def test_parse_item_prices_rejects_empty_input():
    with pytest.raises(ValueError):
        parse_item_prices("")

def test_parse_item_prices_rejects_empty_elements():
    with pytest.raises(ValueError):
        parse_item_prices("100,,200")

def test_parse_item_prices_rejects_invalid_types():
    with pytest.raises(TypeError):
        parse_item_prices(None)
```

B.4 Property Tests for split_amount:

```
# TEST 3: split_amount()
@given(st.integers(min_value=0, max_value=1000000), st.integers(min_value=1, max_value=10000))
def test_split_amount_valid_inputs(total, people):
    text = f"{total}/{people}"
    assert split_amount(text) == total // people

@given(st.integers(max_value=-1))
def test_split_amount_rejects_negative_total(total):
    with pytest.raises(ValueError):
        split_amount(f"{total}/4")

@given(st.integers(max_value=0))
def test_split_amount_rejects_zero_or_negative_people(people):
    with pytest.raises(ValueError):
        split_amount(f"1200/{people}")

@given(st.sampled_from(["", "1200", "/4", "1200/", "1200/4/2", "abc/4", "1200/abc"]))
def test_split_amount_rejects_invalid_formats(text):
    with pytest.raises(ValueError):
        split_amount(text)

def test_split_amount_rejects_invalid_types():
    with pytest.raises(TypeError):
        split_amount(None)
```